

Programación dinámica

Top Down

Joaquín Laks - Ezequiel Companetz - Valeria Pellegrini
Leandro Raffo - Lautaro Lasorsa

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

¿Qué sabemos de Backtracking?

- Explora *sistemáticamente* el espacio de soluciones.

¿Qué sabemos de Backtracking?

- Explora *sistemáticamente* el espacio de soluciones.
- Recuerda toda la trayectoria tomada hasta el momento (el *historial*).

¿Qué sabemos de Backtracking?

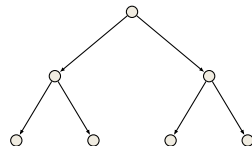
- Explora *sistemáticamente* el espacio de soluciones.
- Recuerda toda la trayectoria tomada hasta el momento (el *historial*).
- Árbol de decisiones con *poda* por factibilidad o cota.

¿Qué sabemos de Backtracking?

- Explora *sistemáticamente* el espacio de soluciones.
- Recuerda toda la trayectoria tomada hasta el momento (el *historial*).
- Árbol de decisiones con *poda* por factibilidad o cota.
- Aplicación típica: mochila, coloreo, n reinas, clique.

¿Qué sabemos de Backtracking?

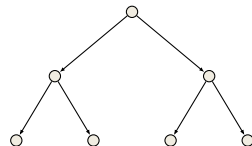
- Explora *sistemáticamente* el espacio de soluciones.
- Recuerda toda la trayectoria tomada hasta el momento (el *historial*).
- Árbol de decisiones con *poda* por factibilidad o cota.
- Aplicación típica: mochila, coloreo, n reinas, clique.
- Complejidad típica: **exponencial** ($\mathcal{O}(2^n)$, $\mathcal{O}(n!)$).



árbol de BT

¿Qué sabemos de Backtracking?

- Explora *sistemáticamente* el espacio de soluciones.
- Recuerda toda la trayectoria tomada hasta el momento (el *historial*).
- Árbol de decisiones con *poda* por factibilidad o cota.
- Aplicación típica: mochila, coloreo, n reinas, clique.
- Complejidad típica: **exponencial** ($\mathcal{O}(2^n)$, $\mathcal{O}(n!)$).



árbol de BT

La pregunta que motiva la clase

¿Podemos explorar ese árbol *pagando menos*?

Programación Dinámica: los 4 pasos

- 1 **Identificar un conjunto minimal de información** que describa un subproblema (el *estado*).

Programación Dinámica: los 4 pasos

- 1 **Identificar un conjunto minimal de información** que describa un subproblema (el *estado*).
- 2 Escribir una **función recursiva** sobre ese estado, apelando al *principio de optimalidad*.

Programación Dinámica: los 4 pasos

- 1 **Identificar un conjunto minimal de información** que describa un subproblema (el *estado*).
- 2 Escribir una **función recursiva** sobre ese estado, apelando al *principio de optimalidad*.
- 3 Observar que hay **superposición de subproblemas**.

Programación Dinámica: los 4 pasos

- 1 **Identificar un conjunto minimal de información** que describa un subproblema (el *estado*).
- 2 Escribir una **función recursiva** sobre ese estado, apelando al *principio de optimalidad*.
- 3 Observar que hay **superposición de subproblemas**.
- 4 **Memorizar los resultados** de los subproblemas para no recalcularlos.

Programación Dinámica: los 4 pasos

- 1 **Identificar un conjunto minimal de información** que describa un subproblema (el *estado*).
- 2 Escribir una **función recursiva** sobre ese estado, apelando al *principio de optimalidad*.
- 3 Observar que hay **superposición de subproblemas**.
- 4 **Memorizar los resultados** de los subproblemas para no recalcularlos.

Qué describe este framework

Es el **modelado** del problema. Es la receta que vamos a aplicar a todo algoritmo de PD.

Los 4 pasos, con nombres propios

Vocabulario

- *Estado*: parámetros que identifican un subproblema.
- *Principio de optimalidad*: la solución óptima se construye con soluciones óptimas de subproblemas.
- *Superposición*: el mismo subproblema aparece muchas veces en el árbol de llamadas. Distintos subproblemas lo tienen como subproblema.
- *Memorización*: recordar para no recalcular.

Los 4 pasos, con nombres propios

Vocabulario

- *Estado*: parámetros que identifican un subproblema.
- *Principio de optimalidad*: la solución óptima se construye con soluciones óptimas de subproblemas.
- *Superposición*: el mismo subproblema aparece muchas veces en el árbol de llamadas. Distintos subproblemas lo tienen como subproblema.
- *Memorización*: recordar para no recalcular.

Qué va a cambiar respecto de BT

- BT explora ramas; PD reaprovecha subproblemas.
- En BT, los parámetros describen una *trayectoria*.
- En PD, el estado debe *olvidar* lo irrelevante.

Kahoot

El ejército del rey

El rey Cambyses está interesado en armar ejércitos en una serie de días consecutivos.

Más aún, le interesa que el número de personas de su ejército en el día i sea equivalente a la suma del número de personas del ejército que formó los días $i - 1$ e $i - 2$.

La excepción es en los días 0 y 1, en los cuales su ejército tendrá siempre una persona.

Dado un día n , queremos calcular el número de personas de su ejército.

Consignas

- 1 Escribir la función recursiva que quiere calcular el rey en términos de $n \in \mathbb{N}$. ¿Es posible que esta función devuelva distintos valores para el mismo n ?
- 2 ¿Qué complejidad tiene esta función si se la implementa de forma directa?
- 3 Dibujar el árbol de llamadas recursivas para el día 4. ¿Se calculan las mismas cosas más de una vez? ¿Cuáles?
- 4 Implementar un algoritmo recursivo que evite calcular dos veces lo mismo, guardando los resultados ya calculados para utilizarlos posteriormente. *Ayuda:* considerar guardar los valores ya calculados en un arreglo de n posiciones y consultarlo para saber si el valor actual ya fue calculado.
- 5 Determinar la complejidad temporal y espacial del nuevo algoritmo. ¿Cómo se compara con la implementación directa?
- 6 Optimizar el algoritmo anterior utilizando un enfoque *bottom-up* para lograr complejidad espacial $O(1)$.

Función recursiva

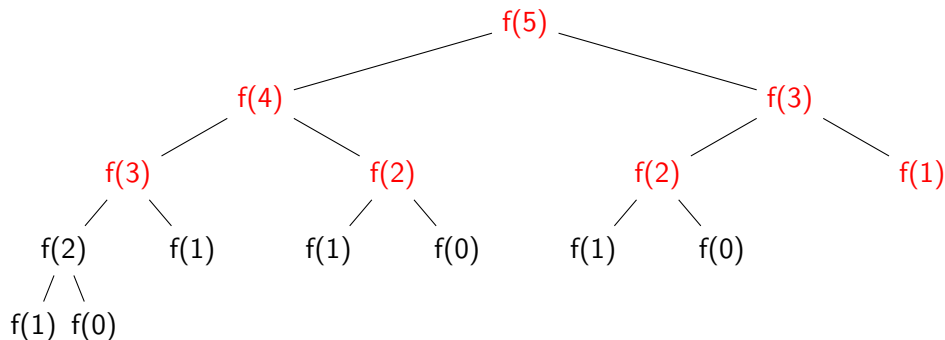
$f(i)$ = cantidad de soldados el día i .

Función recursiva

$$f(i) = \begin{cases} 1 & \text{si } i \leq 1 \\ f(i-1) + f(i-2) & \text{en otro caso} \end{cases}$$

¿La conocen? ¿Cuántos llamados recursivos hace?

Llamados recursivos



Hay un subárbol binario completo de altura $\lfloor n/2 \rfloor$. Entonces, el árbol tiene más de $2^{\lfloor n/2 \rfloor}$ nodos: $\Omega(2^{n/2})$.

Estados vs. llamados recursivos

Las funciones matemáticas tienen una única salida para cada entrada, así que no puede haber más respuestas que formas distintas de llamarla.

¿De cuántas maneras distintas podemos llamar a $f(i)$?

Estados vs. llamados recursivos

Las funciones matemáticas tienen una única salida para cada entrada, así que no puede haber más respuestas que formas distintas de llamarla.

¿De cuántas maneras distintas podemos llamar a $f(i)$? $O(n)$

O vs. Ω

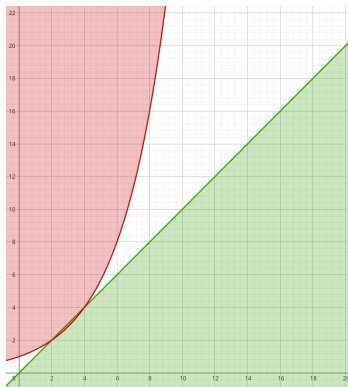


Figura: En rojo $\Omega(2^{n/2})$, en verde $O(n)$.

Solución dinámica

Queremos resolver cada subproblema una sola vez. Para eso usamos una **estructura de memoización**.

Necesitamos una estructura con acceso aleatorio en tiempo constante donde guardar los resultados de cada estado.

Como i puede tomar valores entre 0 y n , podemos usar un arreglo M para guardar $M[i] = f(i)$.

Solución dinámica

Cada vez que necesitemos calcular $f(i)$, primero consultamos si su resultado ya está guardado en M . Si lo está, lo usamos en $O(1)$.

Pseudocódigo sin dinámica

$F(i)$:

- Si $i \leq 1$, devolver 1.
- Si no, devolver $F(i - 1) + F(i - 2)$.

Pseudocódigo con dinámica

- Sea M un arreglo inicializado con valores indefinidos.

$F(i)$:

- Si $M[i]$ está definido, devolver $M[i]$.
- Si $i \leq 1$, devolver 1.
- $M[i] \leftarrow F(i - 1) + F(i - 2)$.
- Devolver $M[i]$.

Nueva complejidad

Los llamados a F cuyos resultados ya están guardados equivalen a una lectura de memoria, que cuesta $O(1)$.

La complejidad temporal es la cantidad de estados por el costo de calcular cada estado. En este caso, $O(n) \cdot O(1) = O(n)$.

¿Perdimos algo a cambio de esta mejora en complejidad temporal?

¿Perdimos algo a cambio de esta mejora en complejidad temporal? Ahora tenemos una **complejidad espacial** de $O(n)$.

Ejercicio 4: Caminos en DAGs

- 1 Proponer un algoritmo que tome tiempo $O(n + m)$ para contar la cantidad de caminos distintos en un DAG entre dos vértices s y t . Demostrar su correctitud y sus complejidades temporal y espacial. **Ayuda:** considerar que la función recursiva calcule la cantidad de caminos distintos que terminan en un vértice dado del DAG.
- 2 Si resolvieron el problema con un enfoque *top-down*, háganlo *bottom-up*, y viceversa. **Ayuda:** para el enfoque bottom-up, computar d de acuerdo a un orden topológico $v_1, \dots, v_n$, donde $v_i \rightarrow v_j$ solo si $i < j$. Este orden se puede computar en $O(n + m)$.

Recursión

Nos dan como ayuda pensar la función recursiva que calcula la cantidad de caminos que termina en un vértice. Supongamos que f es esta función, veamos que tiene que cumplir, primero $f(s) = 1$ que es el camino de s a s que consiste de un solo vértice. Para cualquier otro vértice lo que podemos decir es que la cantidad de caminos distintos de s que llegan a él son la suma de los caminos de s a sus padres. Esto nos da la siguiente función recursiva

$$f(v) = \begin{cases} 1 & \text{si } v = s \\ \sum_{w|w \rightarrow v} f(w) & \text{caso contrario} \end{cases}$$

PD Top down

Intentemos construir un algoritmo de PD top down a partir de esto. Pero antes un par de preguntas

- ¿Cuáles serían nuestros estados?

PD Top down

Intentemos construir un algoritmo de PD top down a partir de esto. Pero antes un par de preguntas

- ¿Cuáles serían nuestros estados?
Cada estado equivale a un vértice v y nos dice cuántos caminos distintos hay de s a v
- ¿Dónde usamos que es un DAG?

PD Top down

Intentemos construir un algoritmo de PD top down a partir de esto. Pero antes un par de preguntas

- ¿Cuáles serían nuestros estados?
Cada estado equivale a un vértice v y nos dice cuántos caminos distintos hay de s a v
- ¿Dónde usamos que es un DAG?
En la recurrencia como nuestro grafo es un DAG nos garantiza que termina y no hay recursión infinita

Algoritmo

```
M = [UNDEF] * n
f(v) {
    if v == s { return 1 }

    if M[v] == UNDEF {
        sum_paths = 0
        for w in N_in(v) {
            sum_paths += f(w)
        }
        M[v] = sum_paths
    }

    return M[v]
}
```

Complejidad

- ¿Cuántos estados posibles hay?

Complejidad

- ¿Cuántos estados posibles hay?

$$O(n)$$

- ¿Cuánto trabajo se hace en cada estado?

Complejidad

- ¿Cuántos estados posibles hay?

$$O(n)$$

- ¿Cuánto trabajo se hace en cada estado?

$$O(N^-(v))$$

- Un análisis naive de la complejidad dónde visitamos todos los estados y sobreacotando $N^-(v)$ por $n - 1$ nos diría que es $O(n^2)$ pero podemos hacer mejor, si lo vemos un poquito mejor, podemos asumir que visitamos todos los vértices, cada uno llama a la recursión si no se visitó antes es decir tenemos

$$O\left(\sum_{v \in V} N^-(v)\right) =$$

$$O(m)$$

Correctitud

Probar que esto es correcto es un poco más difícil que lo que venimos haciendo. A priori queremos usar inducción pero no tenemos forma de “achicar” nada.

Correctitud

Probar que esto es correcto es un poco más difícil que lo que venimos haciendo. A priori queremos usar inducción pero no tenemos forma de “achicar” nada.

- Consideremos entonces que además tenemos el grafo ordenado por su orden topológico y que $s < t$ (caso contrario siempre podemos devolver 0 antes de siquiera correr el algoritmo)
- Vamos a hacer inducción en la posición del orden topológico usando y y vamos a probar el siguiente predicado:

$$P(k) = \#\{\text{camino de } s \text{ a } v_k \text{ en el orden } y\}$$

Caso base

Recordemos que

$$P(k) = \#\{\text{caminos de } s \text{ a } v_k \text{ en el orden } \}$$

Caso base: $k = 1$, podemos separar en dos

- $v_1 = s$, en tal caso nuestra recursión nos dice que hay un solo camino, el que cuenta con un solo vértice
- $v_1 \neq s$ ahora caemos en el segundo caso de nuestra recursión pero no hay antecesores por lo cual devolvemos 0 que es lo esperable ya que no hay caminos $s \rightarrow v_1$.

Caso base

Recordemos que

$$P(k) = \#\{\text{caminos de } s \text{ a } v_k \text{ en el orden } \}$$

Caso inductivo: Vamos a hacer inducción fuerte, supongamos que probamos $P(j)$ para todo $1 \leq j \leq k$ y queremos ver $P(k+1)$.

- Todo camino que llega a v_{k+1} en un DAG tiene que haber pasado por alguno de sus antecesores (si los hay)
- Si w es un antecesor de v_{k+1} entonces dado un orden topológico es alguno de los v_j con $1 \leq j \leq k$, entonces por HI sabemos que $f(w)$ devuelve los caminos distintos de s a w .
- Si sumamos sobre todos los antecesores entonces conseguimos todos los caminos de s a v_{k+1} que es justamente lo que queríamos.

Ejercicio 5: OptiPago

Tenemos un multiconjunto B de valores de billetes y queremos comprar un producto de costo c en una máquina que no da vuelto. Para adquirirlo, debemos cubrir su costo usando un subconjunto de nuestros billetes.

Queremos pagar con el **mínimo exceso posible** y, entre las opciones con igual exceso, usar la **menor cantidad de billetes**.

Ejemplo: Si $c = 14$ y $B = \{2, 3, 5, 10, 20, 20\}$, la solución es pagar 15 con dos billetes: uno de 10 y otro de 5.

Sea $B = \{b_1, \dots, b_n\}$. En cada decisión de nuestro árbol, podemos usar el billete b_n (y restarlo del monto que falta cubrir) o no usarlo.

Consignas del Ejercicio

Modelado y Backtracking

- a) Definir los conjuntos de soluciones candidatas y válidas, y escribir la fórmula que corresponde a la solución del problema.
- b) Definir el conjunto de soluciones parciales. Escribir una función recursiva $\text{OptiPago}(B, c)$, donde (c', q) indica que c' es el mínimo monto $\geq c$ que se puede pagar, y q es la mínima cantidad de billetes. Indicar la llamada inicial y demostrar su correctitud por inducción.
- c) Implementar la función recursiva con parámetros (B, i, j) que compute $\text{OptiPago}(\{b_1, \dots, b_i\}, j)$. Indicar la complejidad del algoritmo.

Consignas del Ejercicio (cont.)

Programación Dinámica

- d) Reescribir OptiPago como una función recursiva $\text{OptiPagoPD}_B(i, j)$, dejando fijo el parámetro B . Determinar cuándo conviene implementarla con memoización y cuándo sin ella, en términos de complejidad temporal.
- e) Adaptar el algoritmo del inciso anterior para incluir una estructura de memoización.

Soluciones candidatas y válidas

Dado el conjunto de billetes $B = \{b_1, b_2, \dots, b_n\}$, podemos modelar una solución como un vector binario $X = (x_1, x_2, \dots, x_n)$ donde $x_i \in \{0, 1\}$ indica si usamos el billete b_i o no.

Soluciones candidatas ($\mathcal{S}$)

Cualquier subconjunto de billetes de B . Formalmente, $\mathcal{S} = \{0, 1\}^n$.

Soluciones candidatas y válidas

Dado el conjunto de billetes $B = \{b_1, b_2, \dots, b_n\}$, podemos modelar una solución como un vector binario $X = (x_1, x_2, \dots, x_n)$ donde $x_i \in \{0, 1\}$ indica si usamos el billete b_i o no.

Soluciones candidatas ($\mathcal{S}$)

Cualquier subconjunto de billetes de B. Formalmente, $\mathcal{S} = \{0, 1\}^n$.

Soluciones válidas ($\mathcal{S}'$)

Son aquellas soluciones candidatas $X \in \mathcal{S}$ donde la suma de los billetes alcanza o supera el costo del producto C :

$$\mathcal{S}' = \left\{ X \in \mathcal{S} \mid \sum_{i=1}^n x_i \cdot b_i \geq C \right\}$$

Función objetivo

Queremos minimizar una tupla formada por el Exceso y la Cantidad de billetes, buscando siempre dentro de nuestro conjunto de soluciones válidas $\mathcal{S}'$.

Para todo $X \in \mathcal{S}'$, buscamos el mínimo global:

$$\min_{X \in \mathcal{S}'} \left(\sum_{i=1}^n x_i \cdot b_i - C, \sum_{i=1}^n x_i \right)$$

Relación de orden: Dadas dos soluciones válidas $(c_1, q_1), (c_2, q_2)$ definimos

$$\text{mín}_{opt}((c_1, q_1), (c_2, q_2)) = \begin{cases} (c_1, q_1) & \text{si } c_1 < c_2 \vee (c_1 = c_2 \wedge q_1 \leq q_2) \\ (c_2, q_2) & \text{en caso contrario} \end{cases}$$

Soluciones parciales y subproblemas

Soluciones parciales ($\mathcal{P}$)

El conjunto de todas las secuencias de decisiones tomadas sobre un prefijo de k billetes ($0 \leq k \leq n$). Formalmente: $\mathcal{P} = \bigcup_{k=0}^n \{0, 1\}^k$

Una solución parcial individual es un elemento $X_k = (x_1, \dots, x_k) \in \mathcal{P}$.

Soluciones parciales y subproblemas

Soluciones parciales ($\mathcal{P}$)

El conjunto de todas las secuencias de decisiones tomadas sobre un prefijo de k billetes ($0 \leq k \leq n$). Formalmente: $\mathcal{P} = \bigcup_{k=0}^n \{0, 1\}^k$

Una solución parcial individual es un elemento $X_k = (x_1, \dots, x_k) \in \mathcal{P}$.

Sucesoras de una solución parcial

A partir de una solución parcial X_k , en el siguiente paso de la exploración tomamos una decisión sobre el billete b_{k+1} . Las sucesoras válidas en el árbol de búsqueda son las tuplas de tamaño $k + 1$:

- $X_{k+1} = (x_1, \dots, x_k, 1)$ (agregamos el billete b_{k+1} a la solución)
- $X_{k+1} = (x_1, \dots, x_k, 0)$ (no agregamos el billete b_{k+1})

Función recursiva

Recurrencia

Sea $B = \{b_1, \dots, b_n\}$ y $B' = B \setminus \{b_n\}$. La función devuelve la tupla (c', q) óptima:

$$\text{OptiPago}(B, c) = \begin{cases} (0, 0) & \text{si } c \leq 0 \\ (\infty, \infty) & \text{si } c > 0 \text{ y } B = \emptyset \\ \text{mín}_{\text{opt}} \left(\text{OptiPago}(B', c), \text{OptiPago}(B', c - b_n) \oplus b_n \right) & \text{cc} \end{cases}$$

Donde $(c', q) \oplus b_n = (c' + b_n, q + 1)$, y el operador mín_{opt} devuelve la tupla que minimiza la primera componente (costo total) y desempata minimizando la segunda (cantidad de billetes).

Función recursiva

Recurrencia

Sea $B = \{b_1, \dots, b_n\}$ y $B' = B \setminus \{b_n\}$. La función devuelve la tupla (c', q) óptima:

$$\text{OptiPago}(B, c) = \begin{cases} (0, 0) & \text{si } c \leq 0 \\ (\infty, \infty) & \text{si } c > 0 \text{ y } B = \emptyset \\ \text{mín}_{\text{opt}} \left(\text{OptiPago}(B', c), \text{OptiPago}(B', c - b_n) \oplus b_n \right) & \text{cc} \end{cases}$$

Donde $(c', q) \oplus b_n = (c' + b_n, q + 1)$, y el operador mín_{opt} devuelve la tupla que minimiza la primera componente (costo total) y desempata minimizando la segunda (cantidad de billetes).

Llamada inicial

Para resolver el problema con un conjunto de billetes original B y un costo C , invocamos:

$$\text{OptiPago}(B, C)$$

Demostración de Correctitud

Propiedad $P(n)$

Para todo $c \in \mathbb{R}$, $\text{OptiPago}(B, c)$ devuelve la tupla óptima (c', q) para todo multiconjunto B de tamaño $|B| = n$.

Demostración de Correctitud

Propiedad $P(n)$

Para todo $c \in \mathbb{R}$, $\text{OptiPago}(B, c)$ devuelve la tupla óptima (c', q) para todo multiconjunto B de tamaño $|B| = n$.

Caso Base ($n = 0$)

Tenemos $B = \emptyset$.

- Si $c \leq 0$: El costo ya está cubierto, la respuesta $(0, 0)$ es correcta y no usa billetes extra.
- Si $c > 0$: Es imposible cubrir un saldo positivo sin billetes, devolver (∞, ∞) es correcto.

Por lo tanto, $P(0)$ es verdadera.

Demostración de Correctitud (cont.)

Paso Inductivo ($n > 0$)

Hipótesis Inductiva (HI): Asumimos que $P(n - 1)$ es verdadera.

Sea B tal que $|B| = n$ y tomemos el último billete $b_n \in B$. Llamemos $B' = B \setminus \{b_n\}$. Cualquier subconjunto óptimo se divide en dos casos:

- 1 **No incluye b_n :** El problema se reduce a encontrar el óptimo usando solo B' . Por HI, $\text{OptiPago}(B', c)$ calcula esto correctamente.
- 2 **Sí incluye b_n :** Al usar b_n , el monto restante a cubrir es $c - b_n$. Por HI, $\text{OptiPago}(B', c - b_n)$ devuelve la tupla óptima para ese remanente. El costo total de esta rama es sumarle b_n al costo y 1 a la cantidad de billetes.

Dado que el algoritmo explora ambas ramas y el operador mín_{opt} selecciona estrictamente la mejor tupla según el criterio del problema (mínimo exceso, luego mínima cantidad), la decisión tomada es globalmente óptima. Luego, $P(n - 1) \implies P(n)$. ■

Implementación de OptiPago

Código en Python

```
# B: lista billetes, i: cant. billetes prefijo, j: costo restante
def opti_pago(B, i, j):
    # 1. Casos base
    if j <= 0: return (0, 0)
    if i == 0: return (math.inf, math.inf)

    # 2. Rama 1: No incluimos el billete actual
    exc = opti_pago(B, i - 1, j)

    # 3. Rama 2: Si incluimos el billete actual
    c, q = opti_pago(B, i - 1, j - B[i - 1])
    inc = (c + B[i - 1], q + 1)

    # 4. Retornar el optimo (min compara lexicograficamente)
    return min(exc, inc)
```

Análisis de Complejidad

Complejidad Temporal

El tiempo de ejecución en el peor caso es $\mathcal{O}(2^n)$, donde n es la cantidad total de billetes.

Justificación: En el peor caso (por ejemplo, si la suma de todos los billetes no alcanza para cubrir el costo C , o los billetes son muy chicos), por cada billete disponible se exploran siempre las dos decisiones recursivas, generando un árbol binario de llamadas de altura n . La cantidad total de nodos es proporcional a 2^n , y en cada llamada se realiza trabajo constante $\mathcal{O}(1)$.

Análisis de Complejidad

Complejidad Temporal

El tiempo de ejecución en el peor caso es $\mathcal{O}(2^n)$, donde n es la cantidad total de billetes.

Justificación: En el peor caso (por ejemplo, si la suma de todos los billetes no alcanza para cubrir el costo C , o los billetes son muy chicos), por cada billete disponible se exploran siempre las dos decisiones recursivas, generando un árbol binario de llamadas de altura n . La cantidad total de nodos es proporcional a 2^n , y en cada llamada se realiza trabajo constante $\mathcal{O}(1)$.

Complejidad Espacial

La complejidad espacial extra es $\mathcal{O}(n)$.

Justificación: La memoria adicional consumida corresponde a los registros de activación apilados en el *call stack*. La profundidad máxima de la pila de recursión está acotada por la cantidad máxima de decisiones secuenciales antes de alcanzar un caso base, que es exactamente n .

OptiPago con B global

Recurrencia reescrita

Asumiendo el multiconjunto $B = \{b_1, \dots, b_n\}$ accesible globalmente, el estado se define solo por i (cantidad de billetes del prefijo) y j (monto restante a cubrir).

$$\text{OptiPago}'_B(i, j) = \begin{cases} (0, 0) & \text{si } j \leq 0 \\ (\infty, \infty) & \text{si } j > 0 \text{ y } i = 0 \\ \min_{opt} \left(\text{OptiPago}'_B(i-1, j), \right. & \text{c.c.} \\ \left. \text{OptiPago}'_B(i-1, j-b_i) \oplus b_i \right) & \end{cases}$$

Donde $(c', q) \oplus b_i = (c' + b_i, q + 1)$.

OptiPago con B global

Recurrencia reescrita

Asumiendo el multiconjunto $B = \{b_1, \dots, b_n\}$ accesible globalmente, el estado se define solo por i (cantidad de billetes del prefijo) y j (monto restante a cubrir).

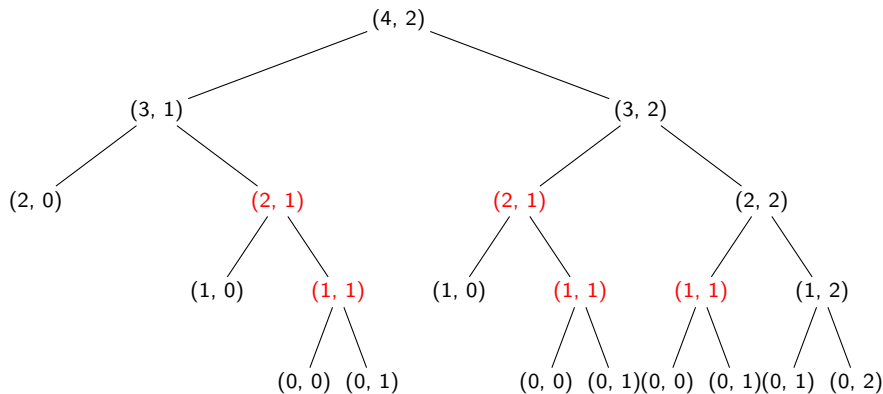
$$\text{OptiPago}'_B(i, j) = \begin{cases} (0, 0) & \text{si } j \leq 0 \\ (\infty, \infty) & \text{si } j > 0 \text{ y } i = 0 \\ \min_{opt} \left(\text{OptiPago}'_B(i-1, j), \right. & \text{c.c.} \\ \left. \text{OptiPago}'_B(i-1, j - b_i) \oplus b_i \right) & \end{cases}$$

Donde $(c', q) \oplus b_i = (c' + b_i, q + 1)$.

Llamada inicial

Para resolver el problema original teniendo n billetes en total y un costo objetivo C , la llamada inicial es: $\text{OptiPago}'_B(n, C)$

Árbol de recursión (Backtracking)



Ejecución real asumiendo $B = \{1, 1, 1, 1\}$ y $C = 2$. **Notamos masivo solapamiento de subproblemas (nodos en rojo).**

Algoritmo Top-Down (Memoización)

Adaptación con estructura de memoización en Python

```
# memo: matriz (n + 1) x (C + 1) inicializada en None
def opti_memo(B, i, j, memo):
    # 1. Casos base
    if j <= 0: return (0, 0)
    if i == 0: return (math.inf, math.inf)
    # 2. Consultar memoria
    if memo[i][j] is not None:
        return memo[i][j]
    # 3. Explorar ramas recursivas
    exc = opti_memo(B, i - 1, j, memo)
    c, q = opti_memo(B, i - 1, j - B[i - 1], memo)
    inc = (c + B[i - 1], q + 1)
    # 4. Guardar y retornar el optimo
    memo[i][j] = min(exc, inc)
    return memo[i][j]
```

Comparación: Backtracking puro vs. Memoización

Sin Memoización

```
def opti_pago(B, i, j):  
    if j <= 0: return (0, 0)  
    if i == 0:  
        return (math.inf, math.inf)  
  
    exc = opti_pago(B, i-1, j)  
  
    c, q = opti_pago(B, i-1,  
                    j - B[i-1])  
    inc = (c + B[i-1], q + 1)  
  
    return min(exc, inc)
```

Con Memoización

```
def opti_memo(B, i, j, memo):  
    if j <= 0: return (0, 0)  
    if i == 0:  
        return (math.inf, math.inf)  
    # 1. Consultar memoria  
    if memo[i][j] is not None:  
        return memo[i][j]  
    exc = opti_memo(B, i-1, j, memo)  
    c, q = opti_memo(B, i-1,  
                    j - B[i-1], memo)  
    inc = (c + B[i-1], q + 1)  
    # 2. Guardar antes de retornar  
    memo[i][j] = min(exc, inc)  
    return memo[i][j]
```

Comparación de Complejidades

Sin Memoización vs Con Memoización

- **Sin memoización (Backtracking puro):** Complejidad temporal $\mathcal{O}(2^n)$. Depende puramente de la cantidad de billetes n .
- **Con memoización (Programación Dinámica):** Los estados posibles a guardar combinan $i \in \{0, \dots, n\}$ y $j \in \{1, \dots, C\}$. El cálculo por estado es $\mathcal{O}(1)$. La complejidad temporal resulta $\mathcal{O}(n \cdot C)$.

Comparación de Complejidades

Sin Memoización vs Con Memoización

- **Sin memoización (Backtracking puro):** Complejidad temporal $\mathcal{O}(2^n)$. Depende puramente de la cantidad de billetes n .
- **Con memoización (Programación Dinámica):** Los estados posibles a guardar combinan $i \in \{0, \dots, n\}$ y $j \in \{1, \dots, C\}$. El cálculo por estado es $\mathcal{O}(1)$. La complejidad temporal resulta $\mathcal{O}(n \cdot C)$.

Cuándo conviene usar cada estrategia

- **Memoización es mejor** cuando el costo objetivo C es relativamente pequeño o acotado, de forma que $n \cdot C \ll 2^n$. En la práctica, el solapamiento de subproblemas es masivo y la tabla salva el rendimiento.
- **Sin memoización es mejor** cuando el monto C es gigantesco (crece exponencialmente respecto a n , por ejemplo $C \approx 2^n$). Acá, la tabla de $n \times C$ resulta inmanejable en tiempo y memoria, haciendo que la exploración exhaustiva $\mathcal{O}(2^n)$ sea preferible.

Las legiones del César

Las legiones del César

Programación Dinámica Top-Down

Enunciado: Ejercicio 13

Este problema fue extraído de Codeforces 118D.

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos. Tiene P patos y D dodos, y quiere formar una fila tal que:

Enunciado: Ejercicio 13

Este problema fue extraído de Codeforces 118D.

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos. Tiene P patos y D dodos, y quiere formar una fila tal que:

- No haya más de m_P patos consecutivos.

Enunciado: Ejercicio 13

Este problema fue extraído de Codeforces 118D.

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos. Tiene P patos y D dodos, y quiere formar una fila tal que:

- No haya más de m_P patos consecutivos.
- No haya más de m_D dodos consecutivos.

Enunciado: Ejercicio 13

Este problema fue extraído de Codeforces 118D.

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos. Tiene P patos y D dodos, y quiere formar una fila tal que:

- No haya más de m_P patos consecutivos.
- No haya más de m_D dodos consecutivos.
- Los patos sean indistinguibles entre sí, al igual que los dodos.

Enunciado: Ejercicio 13

Este problema fue extraído de Codeforces 118D.

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos. Tiene P patos y D dodos, y quiere formar una fila tal que:

- No haya más de m_P patos consecutivos.
- No haya más de m_D dodos consecutivos.
- Los patos sean indistinguibles entre sí, al igual que los dodos.

Le interesa contar la cantidad de formas posibles de formar esta fila.

Ejercicio 13: Consignas

- 1 Definir una función recursiva $f_{m_P, m_D}(t, n_P, n_D, k_P, k_D)$ que calcule la cantidad de formas de poner los patos y dodos restantes, dado que quedan t lugares, n_P patos y n_D dodos, y que al final de la fila formada hasta ahora hay k_P patos y k_D dodos consecutivos. Pensar qué elección tomar en cada paso.

Ejercicio 13: Consignas

- 1 Definir una función recursiva $f_{m_P, m_D}(t, n_P, n_D, k_P, k_D)$ que calcule la cantidad de formas de poner los patos y dodos restantes, dado que quedan t lugares, n_P patos y n_D dodos, y que al final de la fila formada hasta ahora hay k_P patos y k_D dodos consecutivos. Pensar qué elección tomar en cada paso.
- 2 Pensar cómo modificar la función para que su firma sea $f_{m_P, m_D}(n_P, n_D, k, ultTropa)$, donde *ultTropa* indica cuál fue la última tropa puesta y k la cantidad consecutiva de esa tropa.

Ejercicio 13: Consignas

- 1 Definir una función recursiva $f_{m_P, m_D}(t, n_P, n_D, k_P, k_D)$ que calcule la cantidad de formas de poner los patos y dodos restantes, dado que quedan t lugares, n_P patos y n_D dodos, y que al final de la fila formada hasta ahora hay k_P patos y k_D dodos consecutivos. Pensar qué elección tomar en cada paso.
- 2 Pensar cómo modificar la función para que su firma sea $f_{m_P, m_D}(n_P, n_D, k, ultTropa)$, donde *ultTropa* indica cuál fue la última tropa puesta y k la cantidad consecutiva de esa tropa.
- 3 Pensar cómo modificar la función para que su firma sea $f_{m_P, m_D}(n_P, n_D, ultTropa)$.

Ejercicio 13: Consignas

- 4 Calcular la complejidad temporal y espacial de cada una de las versiones de la función.
- 5 Demostrar la correctitud de cada una de las versiones de la función.

Datos iniciales

Patos: $P = 3$, $MP = 2$



Dodos: $D = 2$, $MD = 1$



Reglas

Hay que ubicar los 3 patos y los 2 dodos; no puede haber más de 2 patos ni más de 1 dodo consecutivos.

Combinación inválida: sobran dodos



D P D P D P

Combinación inválida: sobran dodos



$D P D P D P$

Error: cantidad de dodos

La fila tiene 3 dodos, pero solo hay $D = 2$. Los 3 patos disponibles sí están ubicados.

Las rachas cumplen los máximos; el problema es que no respeta las cantidades disponibles.

Combinación inválida: racha demasiado larga



P P P D D

Combinación inválida: racha demasiado larga



P P P D D

Error: consecutivos

Hay 3 patos seguidos, pero $MP = 2$. En esta fila también quedan 2 dodos seguidos, aunque $MD = 1$.

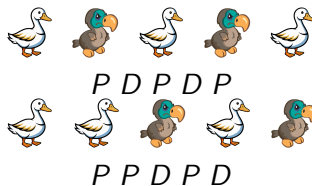
Combinaciones válidas

Con $MP = 2$ y $MD = 1$, estas filas usan exactamente los 3 patos y los 2 dodos, sin superar ningún máximo:



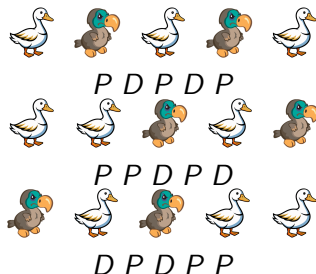
Combinaciones válidas

Con $MP = 2$ y $MD = 1$, estas filas usan exactamente los 3 patos y los 2 dodos, sin superar ningún máximo:



Combinaciones válidas

Con $MP = 2$ y $MD = 1$, estas filas usan exactamente los 3 patos y los 2 dodos, sin superar ningún máximo:



Modelando el estado

Queremos contar todas las formas válidas de completar la fila.

Para decidir qué podemos agregar, necesitamos saber:

- Cuántos patos quedan por ubicar.

Modelando el estado

Queremos contar todas las formas válidas de completar la fila.

Para decidir qué podemos agregar, necesitamos saber:

- Cuántos patos quedan por ubicar.
- Cuántos dodos quedan por ubicar.

Modelando el estado

Queremos contar todas las formas válidas de completar la fila.

Para decidir qué podemos agregar, necesitamos saber:

- Cuántos patos quedan por ubicar.
- Cuántos dodos quedan por ubicar.
- Cuántos lugares quedan disponibles.

Modelando el estado

Queremos contar todas las formas válidas de completar la fila.

Para decidir qué podemos agregar, necesitamos saber:

- Cuántos patos quedan por ubicar.
- Cuántos dodos quedan por ubicar.
- Cuántos lugares quedan disponibles.
- Qué racha termina el prefijo ya construido y cuál es su longitud.

Modelando el estado

Queremos contar todas las formas válidas de completar la fila.

Para decidir qué podemos agregar, necesitamos saber:

- Cuántos patos quedan por ubicar.
- Cuántos dodos quedan por ubicar.
- Cuántos lugares quedan disponibles.
- Qué racha termina el prefijo ya construido y cuál es su longitud.

Una primera firma posible es

$$g(t, n_P, n_D, k_P, k_D),$$

donde k_P y k_D son las longitudes de las rachas finales de patos y dodos. A lo sumo uno de ellos puede ser distinto de cero.

Primera formulación

En cada paso elegimos qué tropa agregar a la derecha del prefijo.

- Podemos agregar un pato si queda alguno y la racha actual de patos tiene longitud menor que m_P .

Primera formulación

En cada paso elegimos qué tropa agregar a la derecha del prefijo.

- Podemos agregar un pato si queda alguno y la racha actual de patos tiene longitud menor que m_P .
- Podemos agregar un dodo si queda alguno y la racha actual de dodos tiene longitud menor que m_D .

Primera formulación

En cada paso elegimos qué tropa agregar a la derecha del prefijo.

- Podemos agregar un pato si queda alguno y la racha actual de patos tiene longitud menor que m_P .
- Podemos agregar un dodo si queda alguno y la racha actual de dodos tiene longitud menor que m_D .

Al agregar una tropa, disminuye en uno la cantidad restante de ese tipo y se actualizan los contadores de rachas. La cantidad total de lugares restantes es $t = n_P + n_D$; por ahora lo conservamos en la firma para escribir la función original y luego veremos que es redundante.

Error frecuente

No alcanza con recordar cuántas tropas quedan: también hay que conocer la racha final para decidir si se puede agregar otro pato o dodo.

Función original: caso base

Usamos la firma original

$$g(t, n_P, n_D, k_P, k_D),$$

donde t es la cantidad de lugares restantes y k_P, k_D son las longitudes de las rachas finales.

Función original: caso base

Usamos la firma original

$$g(t, n_P, n_D, k_P, k_D),$$

donde t es la cantidad de lugares restantes y k_P, k_D son las longitudes de las rachas finales. Si no quedan lugares, la fila ya está completa. Como en los estados alcanzables $t = n_P + n_D$, también se cumple que no quedan tropas:

$$g(0, 0, 0, k_P, k_D) = 1.$$

¿Por qué devuelve 1?

Estamos contando completaciones. Una fila terminada tiene exactamente una continuación: no agregar nada.

Función original: agregar un pato

Podemos agregar un pato si queda alguno y la racha actual de patos todavía no alcanzó su máximo:

$$n_P > 0 \quad \wedge \quad k_P < m_P.$$

Función original: agregar un pato

Podemos agregar un pato si queda alguno y la racha actual de patos todavía no alcanzó su máximo:

$$n_P > 0 \quad \wedge \quad k_P < m_P.$$

Llamamos ponerPato al aporte de esta decisión:

$$\text{ponerPato}(t, n_P, n_D, k_P, k_D) = \begin{cases} g(t-1, n_P-1, n_D, k_P+1, 0) & n_P > 0 \wedge k_P < m_P, \\ 0 & \text{si no.} \end{cases}$$

Si no se puede agregar

Si no queda ningún pato o la racha ya llegó a m_P , esta opción aporta 0 completaciones.

Función original: agregar un dodo

Simétricamente, podemos agregar un dodo si queda alguno y la racha actual de dodos no alcanzó su máximo:

$$n_D > 0 \quad \wedge \quad k_D < m_D.$$

Función original: agregar un dodo

Simétricamente, podemos agregar un dodo si queda alguno y la racha actual de dodos no alcanzó su máximo:

$$n_D > 0 \quad \wedge \quad k_D < m_D.$$

Llamamos ponerDodo al aporte de esta decisión:

$$\text{ponerDodo}(t, n_P, n_D, k_P, k_D) = \begin{cases} g(t-1, n_P, n_D-1, 0, k_D+1) & n_D > 0 \wedge k_D < m_D, \\ 0 & \text{si no.} \end{cases}$$

Si no se puede agregar

Si no queda ningún dodo o la racha ya llegó a m_D , esta opción aporta 0 completaciones.

Función original completa

Sumamos explícitamente las dos funciones auxiliares:

$$g(t, n_P, n_D, k_P, k_D) = \begin{cases} 1 & t = 0, \\ \text{ponerPato}(t, n_P, n_D, k_P, k_D) + \text{ponerDodo}(t, n_P, n_D, k_P, k_D) & t > 0. \end{cases}$$

Función original completa

Sumamos explícitamente las dos funciones auxiliares:

$$g(t, n_P, n_D, k_P, k_D) = \begin{cases} 1 & t = 0, \\ \text{ponerPato}(t, n_P, n_D, k_P, k_D) + \text{ponerDodo}(t, n_P, n_D, k_P, k_D) & t > 0. \end{cases}$$

Cada fila no vacía empieza con una de estas dos decisiones, y cada auxiliar devuelve 0 cuando su decisión no está permitida.

Llamado inicial de la función original

Al principio todavía no construimos ningún prefijo:

$$t = P + D, \quad n_P = P, \quad n_D = D, \quad k_P = k_D = 0.$$

Llamado inicial de la función original

Al principio todavía no construimos ningún prefijo:

$$t = P + D, \quad n_P = P, \quad n_D = D, \quad k_P = k_D = 0.$$

Por lo tanto, la respuesta se obtiene con

$$g(P + D, P, D, 0, 0).$$

Los contadores de rachas empiezan en cero porque todavía no hay una última tropa colocada.

Firma de la función con casos recursivos V2

Podemos interpretar k_P y k_D de otra manera:

Nueva interpretación

k_P y k_D indican cuántas tropas más de cada tipo podemos seguir poniendo consecutivamente desde este estado.

Firma de la función con casos recursivos V2

Podemos interpretar k_P y k_D de otra manera:

Nueva interpretación

k_P y k_D indican cuántas tropas más de cada tipo podemos seguir poniendo consecutivamente desde este estado.

Esa capacidad tiene que respetar dos límites:

- No superar el máximo de tropas consecutivas, m_P o m_D .

Firma de la función con casos recursivos V2

Podemos interpretar k_P y k_D de otra manera:

Nueva interpretación

k_P y k_D indican cuántas tropas más de cada tipo podemos seguir poniendo consecutivamente desde este estado.

Esa capacidad tiene que respetar dos límites:

- No superar el máximo de tropas consecutivas, m_P o m_D .
- No superar la cantidad de tropas de ese tipo que queda disponible.

La firma sigue siendo $g_2(t, n_P, n_D, k_P, k_D)$; cambia el significado de k_P y k_D .

V2: cómo reiniciar una capacidad

Al cambiar de tipo, empezamos una racha nueva. Su capacidad no es automáticamente m_P o m_D : también puede que queden menos tropas.

V2: cómo reiniciar una capacidad

Al cambiar de tipo, empezamos una racha nueva. Su capacidad no es automáticamente m_P o m_D : también puede que queden menos tropas.

Si reiniciamos la capacidad de patos, usamos

$$k_P = \min(n_P, m_P).$$

Si reiniciamos la capacidad de dodos, usamos

$$k_D = \min(n_D, m_D).$$

Intuición

La nueva racha puede crecer hasta el menor de estos dos límites: lo que queda de esa tropa y el máximo permitido.

V2: las transiciones con el cambio en rojo

Al poner un pato, consumimos una unidad de la capacidad de patos y reiniciamos la de dodos. Al poner un dodo, ocurre lo simétrico:

$$\text{ponerPato}_2(t, n_P, n_D, k_P, k_D) = \begin{cases} g_2(t-1, n_P-1, n_D, k_P-1, \text{mín}(n_D, m_D)) & n_P > 0 \wedge k_P > 0, \\ 0 & \text{si no,} \end{cases}$$

V2: las transiciones con el cambio en rojo

Al poner un pato, consumimos una unidad de la capacidad de patos y reiniciamos la de dodos. Al poner un dodo, ocurre lo simétrico:

$$\text{ponerPato}_2(t, n_P, n_D, k_P, k_D) = \begin{cases} g_2(t-1, n_P-1, n_D, k_P-1, \text{mín}(n_D, m_D)) & n_P > 0 \wedge k_P > 0, \\ 0 & \text{si no,} \end{cases}$$

$$\text{ponerDodo}_2(t, n_P, n_D, k_P, k_D) = \begin{cases} g_2(t-1, n_P, n_D-1, \text{mín}(n_P, m_P), k_D-1) & n_D > 0 \wedge k_D > 0, \\ 0 & \text{si no.} \end{cases}$$

V2: función completa e inicialización

La función conserva el caso base y suma los aportes de ponerPato y ponerDodo:

$$g_2(t, n_P, n_D, k_P, k_D) = \begin{cases} 1 & t = 0, \\ \text{ponerPato}_2(t, n_P, n_D, k_P, k_D) + \text{ponerDodo}_2(t, n_P, n_D, k_P, k_D) & t > 0. \end{cases}$$

V2: función completa e inicialización

La función conserva el caso base y suma los aportes de ponerPato y ponerDodo:

$$g_2(t, n_P, n_D, k_P, k_D) = \begin{cases} 1 & t = 0, \\ \text{ponerPato}_2(t, n_P, n_D, k_P, k_D) + \text{ponerDodo}_2(t, n_P, n_D, k_P, k_D) & t > 0. \end{cases}$$

Al comienzo todavía no hay una racha formada, así que ambas capacidades se inicializan con el mínimo entre la cantidad disponible y el máximo:

$$g_2(P + D, P, D, \min(P, m_P), \min(D, m_D)).$$

La primera solución no pasa el time limit

La V2 expresa mejor los límites de cada racha, pero si ejecutamos la recursión directamente y sin memoización, cada tropa todavía puede abrir hasta dos ramas: agregar un pato o agregar un dodo.

La primera solución no pasa el time limit

La V2 expresa mejor los límites de cada racha, pero si ejecutamos la recursión directamente y sin memoización, cada tropa todavía puede abrir hasta dos ramas: agregar un pato o agregar un dodo. La profundidad es $P + D$, así que el árbol puede tener $O(2^{P+D})$ llamados. Para los tamaños de entrada del problema de Codeforces, esta exploración no pasa el time limit.

La primera solución no pasa el time limit

La V2 expresa mejor los límites de cada racha, pero si ejecutamos la recursión directamente y sin memoización, cada tropa todavía puede abrir hasta dos ramas: agregar un pato o agregar un dodo. La profundidad es $P + D$, así que el árbol puede tener $O(2^{P+D})$ llamados. Para los tamaños de entrada del problema de Codeforces, esta exploración no pasa el time limit.

¿Alcanza con agregar memoización? No necesariamente. Aun quitando el parámetro redundante t , la tabla de los otros cuatro parámetros puede tener hasta

$$O((P + 1)(D + 1)(m_P + 1)(m_D + 1))$$

estados. En esta formulación, ese volumen puede exceder el tiempo o la memoria disponibles y dar TLE o MLE.

Motivación para mejorar

Memoizar evita recalcular un mismo estado, pero no reduce cuántos estados distintos guardamos. Vamos a reducir la información del estado.

Primera reducción: eliminar t

La $V2$ todavía guarda cinco datos:

$$g_2(t, n_P, n_D, k_P, k_D).$$

Pero en todos los estados alcanzables, los lugares que quedan son exactamente las tropas que quedan:

$$t = n_P + n_D.$$

Primera reducción: eliminar t

La V2 todavía guarda cinco datos:

$$g_2(t, n_P, n_D, k_P, k_D).$$

Pero en todos los estados alcanzables, los lugares que quedan son exactamente las tropas que quedan:

$$t = n_P + n_D.$$

Entonces t no aporta información nueva. Podemos quitarlo de la firma y trabajar con

$$g'_2(n_P, n_D, k_P, k_D).$$

Idea para reducir un estado

Si un parámetro se puede calcular a partir de los demás, no hace falta almacenarlo.

Reduciendo estados: una capacidad activa

En la V2 guardamos k_P y k_D , pero en cada prefijo una de esas capacidades se puede calcular directamente: es la capacidad de empezar una racha nueva.

- Si la última tropa fue un pato, $k_D = \min(n_D, m_D)$.

Reduciendo estados: una capacidad activa

En la V2 guardamos k_P y k_D , pero en cada prefijo una de esas capacidades se puede calcular directamente: es la capacidad de empezar una racha nueva.

- Si la última tropa fue un pato, $k_D = \min(n_D, m_D)$.
- Si la última tropa fue un dodo, $k_P = \min(n_P, m_P)$.

Solo necesitamos guardar la capacidad de continuar la racha actual y el tipo de tropa al que corresponde:

$$h(n_P, n_D, k, ultTropa), \quad ultTropa \in \{P, D\}.$$

Qué guarda k

k es la cantidad de tropas más del tipo $ultTropa$ que todavía podemos poner consecutivamente.

Al repetir el tipo, k disminuye en uno. Al cambiar de tipo, la nueva capacidad se calcula como $\min(\text{cantidad restante}, \text{máximo permitido})$.

Pequeños pasos vs. grandes pasos

En una recursión, la variable que mide lo que falta puede disminuir de distintas maneras:

- **Pasos pequeños:** disminuirla de a 1, resolviendo una tropa por llamada.

Pequeños pasos vs. grandes pasos

En una recursión, la variable que mide lo que falta puede disminuir de distintas maneras:

- **Pasos pequeños:** disminuirla de a a 1, resolviendo una tropa por llamada.
- **Pasos grandes:** disminuirla en $1, 2, \dots, X$, probando todos los saltos posibles.

Pequeños pasos vs. grandes pasos

En una recursión, la variable que mide lo que falta puede disminuir de distintas maneras:

- **Pasos pequeños:** disminuirla de a a 1, resolviendo una tropa por llamada.
- **Pasos grandes:** disminuirla en $1, 2, \dots, X$, probando todos los saltos posibles.

Intuición

A veces, probar un bloque completo de una vez hace más simple la función recursiva, aunque cada estado tenga más alternativas.

Reduciendo estados: rachas completas

Hasta ahora agregábamos una sola tropa por llamada. ¿Es necesario? No: podemos decidir directamente la longitud de la próxima racha y después cambiar de tipo.

- Si la última racha fue de patos, elegimos cuántos dodos consecutivos agregar, entre 1 y $\min(n_D, m_D)$.

Reduciendo estados: rachas completas

Hasta ahora agregábamos una sola tropa por llamada. ¿Es necesario? No: podemos decidir directamente la longitud de la próxima racha y después cambiar de tipo.

- Si la última racha fue de patos, elegimos cuántos dodos consecutivos agregar, entre 1 y $\text{mín}(n_D, m_D)$.
- Si la última racha fue de dodos, elegimos cuántos patos consecutivos agregar, entre 1 y $\text{mín}(n_P, m_P)$.

Reduciendo estados: rachas completas

Hasta ahora agregábamos una sola tropa por llamada. ¿Es necesario? No: podemos decidir directamente la longitud de la próxima racha y después cambiar de tipo.

- Si la última racha fue de patos, elegimos cuántos dodos consecutivos agregar, entre 1 y $\text{mín}(n_D, m_D)$.
- Si la última racha fue de dodos, elegimos cuántos patos consecutivos agregar, entre 1 y $\text{mín}(n_P, m_P)$.

Después de agregar esa racha, el tipo necesariamente cambia. Como probamos directamente todos los tamaños posibles, ya no hace falta guardar k : solo necesitamos el tipo *ultTropa* de la última racha. La firma final será $f(n_P, n_D, \text{ultTropa})$.

Intercambio

Quitamos el parámetro k , pero cada transición ahora suma los resultados de varias longitudes posibles de racha.

Qué significa la función final

Definimos

$$f(n_P, n_D, ultTropa)$$

como la cantidad de formas válidas de ubicar los n_P patos y los n_D dodos restantes, sabiendo que la última racha ya colocada fue del tipo *ultTropa*.

Qué significa la función final

Definimos

$$f(n_P, n_D, ultTropa)$$

como la cantidad de formas válidas de ubicar los n_P patos y los n_D dodos restantes, sabiendo que la última racha ya colocada fue del tipo *ultTropa*.

El parámetro *ultTropa* puede ser *P* (patos) o *D* (dodos). La próxima racha debe ser del tipo opuesto.

Semántica

$f(n_P, n_D, ultTropa)$ cuenta continuaciones válidas a partir de este estado, no filas completas desde el principio.

Caso base

Si no quedan tropas de ninguno de los dos tipos, existe exactamente una forma de completar la fila: no agregar nada.

Caso base

Si no quedan tropas de ninguno de los dos tipos, existe exactamente una forma de completar la fila: no agregar nada.

$$f(0, 0, ultTropa) = 1.$$

Error frecuente al contar

El caso base devuelve 1, no 0: estamos contando completaciones y existe una forma de completar una fila ya terminada, que es no agregar nada.

Caso base

Si no quedan tropas de ninguno de los dos tipos, existe exactamente una forma de completar la fila: no agregar nada.

$$f(0, 0, ultTropa) = 1.$$

Error frecuente al contar

El caso base devuelve 1, no 0: estamos contando completaciones y existe una forma de completar una fila ya terminada, que es no agregar nada.

Si quedan tropas pero ninguna puede ir a continuación, la suma de opciones es vacía y vale 0.

Caso recursivo: la última racha fue de patos

Si la última racha fue de patos, la siguiente debe ser de dodos.

Caso recursivo: la última racha fue de patos

Si la última racha fue de patos, la siguiente debe ser de dodos.
Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_D, m_D).$$

Fijada una longitud i , quedan n_P patos y $n_D - i$ dodos, y la última racha pasa a ser de dodos.
Por eso:

Caso recursivo: la última racha fue de patos

Si la última racha fue de patos, la siguiente debe ser de dodos.
Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_D, m_D).$$

Fijada una longitud i , quedan n_P patos y $n_D - i$ dodos, y la última racha pasa a ser de dodos.
Por eso:

$$f(n_P, n_D, P) = \sum_{i=1}^{\text{mín}(n_D, m_D)} f(n_P, n_D - i, D).$$

Caso recursivo: la última racha fue de patos

Si la última racha fue de patos, la siguiente debe ser de dodos.
Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_D, m_D).$$

Fijada una longitud i , quedan n_P patos y $n_D - i$ dodos, y la última racha pasa a ser de dodos.
Por eso:

$$f(n_P, n_D, P) = \sum_{i=1}^{\text{mín}(n_D, m_D)} f(n_P, n_D - i, D).$$

Caso sin dodos

Si $n_D = 0$, no hay ninguna elección posible y la suma vale 0.

Caso recursivo: la última racha fue de dodos

Si la última racha fue de dodos, la siguiente debe ser de patos.

Caso recursivo: la última racha fue de dodos

Si la última racha fue de dodos, la siguiente debe ser de patos.

Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_P, m_P).$$

Fijada una longitud i , quedan $n_P - i$ patos y n_D dodos, y la última racha pasa a ser de patos. Entonces:

Caso recursivo: la última racha fue de dodos

Si la última racha fue de dodos, la siguiente debe ser de patos.

Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_P, m_P).$$

Fijada una longitud i , quedan $n_P - i$ patos y n_D dodos, y la última racha pasa a ser de patos. Entonces:

$$f(n_P, n_D, D) = \sum_{i=1}^{\text{mín}(n_P, m_P)} f(n_P - i, n_D, P).$$

Caso recursivo: la última racha fue de dodos

Si la última racha fue de dodos, la siguiente debe ser de patos.
Podemos agregar una racha de longitud

$$i = 1, 2, \dots, \text{mín}(n_P, m_P).$$

Fijada una longitud i , quedan $n_P - i$ patos y n_D dodos, y la última racha pasa a ser de patos.
Entonces:

$$f(n_P, n_D, D) = \sum_{i=1}^{\text{mín}(n_P, m_P)} f(n_P - i, n_D, P).$$

Caso sin patos

Si $n_P = 0$, la suma es vacía y vale 0.

Función recursiva completa

Juntando el caso base y los dos casos recursivos:

Función recursiva completa

Juntando el caso base y los dos casos recursivos:

$$f(n_P, n_D, ultTropa) = \begin{cases} 1, & n_P = 0 \wedge n_D = 0, \\ \sum_{i=1}^{\min(n_D, m_D)} f(n_P, n_D - i, D), & ultTropa = P, \\ \sum_{i=1}^{\min(n_P, m_P)} f(n_P - i, n_D, P), & ultTropa = D \end{cases}$$

Función recursiva completa

Juntando el caso base y los dos casos recursivos:

$$f(n_P, n_D, ultTropa) = \begin{cases} 1, & n_P = 0 \wedge n_D = 0, \\ \sum_{i=1}^{\min(n_D, m_D)} f(n_P, n_D - i, D), & ultTropa = P, \\ \sum_{i=1}^{\min(n_P, m_P)} f(n_P - i, n_D, P), & ultTropa = D \end{cases}$$

Convención

Las sumas vacías valen 0: no hay ninguna continuación que respete el tipo obligatorio de la siguiente racha.

Semántica de la función

$f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas de la fila, no las filas completas desde el principio.

Semántica de la función

$f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas de la fila, no las filas completas desde el principio.

- Si no quedan tropas, hay una única continuación vacía.

Semántica de la función

$f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas de la fila, no las filas completas desde el principio.

- Si no quedan tropas, hay una única continuación vacía.
- Si $ultTropa = P$, agrupamos las continuaciones según la longitud de su próxima racha de dodos.

Semántica de la función

$f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas de la fila, no las filas completas desde el principio.

- Si no quedan tropas, hay una única continuación vacía.
- Si $ultTropa = P$, agrupamos las continuaciones según la longitud de su próxima racha de dodos.
- Si $ultTropa = D$, agrupamos las continuaciones según la longitud de su próxima racha de patos.

Semántica de la función

$f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas de la fila, no las filas completas desde el principio.

- Si no quedan tropas, hay una única continuación vacía.
- Si $ultTropa = P$, agrupamos las continuaciones según la longitud de su próxima racha de dodos.
- Si $ultTropa = D$, agrupamos las continuaciones según la longitud de su próxima racha de patos.

Invariante de la recurrencia

Cada elección fija una longitud única para la próxima racha y deja un subproblema con la misma interpretación.

Llamado que resuelve el problema

Al principio quedan P patos y D dodos, pero todavía no hay una última racha.

Llamado que resuelve el problema

Al principio quedan P patos y D dodos, pero todavía no hay una última racha. Separamos las filas según el tipo de su primera racha:

$$f(P, D, P) + f(P, D, D).$$

Llamado que resuelve el problema

Al principio quedan P patos y D dodos, pero todavía no hay una última racha. Separamos las filas según el tipo de su primera racha:

$$f(P, D, P) + f(P, D, D).$$

En $f(P, D, P)$, la última racha ficticia es de patos, así que la primera racha real debe ser de dodos. En $f(P, D, D)$ ocurre lo contrario: la primera racha real es de patos.

Por qué se suman

Las alternativas son disjuntas y cubren todas las filas. Si $P = D = 0$, la fila vacía se cuenta una sola vez y la respuesta es 1.

Error frecuente

No interpretar *ultTropa* como la primera tropa real: representa la última racha ficticia usada para habilitar el primer bloque.

Correctitud: afirmación y caso base

Probamos por inducción en $n_P + n_D$ que $f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas para todo estado $(n_P, n_D, ultTropa)$.

Correctitud: afirmación y caso base

Probamos por inducción en $n_P + n_D$ que $f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas para todo estado $(n_P, n_D, ultTropa)$.

Caso base: si $n_P + n_D = 0$, entonces $n_P = n_D = 0$. No queda ninguna decisión por tomar y hay exactamente una continuación: la fila vacía. La recurrencia devuelve 1, como corresponde.

Correctitud: afirmación y caso base

Probamos por inducción en $n_P + n_D$ que $f(n_P, n_D, ultTropa)$ cuenta las continuaciones válidas para todo estado $(n_P, n_D, ultTropa)$.

Caso base: si $n_P + n_D = 0$, entonces $n_P = n_D = 0$. No queda ninguna decisión por tomar y hay exactamente una continuación: la fila vacía. La recurrencia devuelve 1, como corresponde.

Hipótesis inductiva: suponemos que la función cuenta correctamente las continuaciones para todo estado con menos de $n_P + n_D$ tropas restantes.

Correctitud: caso $ultTropa = P$

Supongamos que quedan tropas y que la última racha colocada fue de patos. Toda continuación válida debe empezar con una racha de dodos.

Correctitud: caso $ultTropa = P$

Supongamos que quedan tropas y que la última racha colocada fue de patos. Toda continuación válida debe empezar con una racha de dodos.

Su longitud i tiene que cumplir

$$1 \leq i \leq \min(n_D, m_D).$$

Fijada esa longitud, quedan n_P patos y $n_D - i$ dodos, y la última racha pasa a ser de dodos. Por hipótesis inductiva, $f(n_P, n_D - i, D)$ cuenta correctamente esas continuaciones.

Correctitud: caso $ultTropa = P$

Supongamos que quedan tropas y que la última racha colocada fue de patos. Toda continuación válida debe empezar con una racha de dodos.

Su longitud i tiene que cumplir

$$1 \leq i \leq \min(n_D, m_D).$$

Fijada esa longitud, quedan n_P patos y $n_D - i$ dodos, y la última racha pasa a ser de dodos.

Por hipótesis inductiva, $f(n_P, n_D - i, D)$ cuenta correctamente esas continuaciones.

Cada continuación tiene una única longitud para su primera racha de dodos. Por eso las clases de la suma son disjuntas y su unión contiene todas las continuaciones válidas. La fórmula para $ultTropa = P$ es correcta.

Correctitud: caso $ultTropa = D$

Supongamos que quedan tropas y que la última racha colocada fue de dodos. Toda continuación válida debe empezar con una racha de patos.

Correctitud: caso $ultTropa = D$

Supongamos que quedan tropas y que la última racha colocada fue de dodos. Toda continuación válida debe empezar con una racha de patos.

Su longitud i tiene que cumplir

$$1 \leq i \leq \min(n_P, m_P).$$

Fijada esa longitud, quedan $n_P - i$ patos y n_D dodos, y la última racha pasa a ser de patos. Por hipótesis inductiva, $f(n_P - i, n_D, P)$ cuenta correctamente esas continuaciones.

Correctitud: caso $ultTropa = D$

Supongamos que quedan tropas y que la última racha colocada fue de dodos. Toda continuación válida debe empezar con una racha de patos.

Su longitud i tiene que cumplir

$$1 \leq i \leq \min(n_P, m_P).$$

Fijada esa longitud, quedan $n_P - i$ patos y n_D dodos, y la última racha pasa a ser de patos. Por hipótesis inductiva, $f(n_P - i, n_D, P)$ cuenta correctamente esas continuaciones. Cada continuación tiene una única longitud para su primera racha de patos. Luego, las opciones son disjuntas y cubren todas las continuaciones válidas. Esto prueba el caso $ultTropa = D$ y concluye la inducción.

Correctitud del llamado inicial

Si hay al menos una tropa, toda fila completa empieza con exactamente una de estas dos opciones:

- Una racha de dados, contada por $f(P, D, P)$.

Correctitud del llamado inicial

Si hay al menos una tropa, toda fila completa empieza con exactamente una de estas dos opciones:

- Una racha de dodos, contada por $f(P, D, P)$.
- Una racha de patos, contada por $f(P, D, D)$.

Correctitud del llamado inicial

Si hay al menos una tropa, toda fila completa empieza con exactamente una de estas dos opciones:

- Una racha de dodos, contada por $f(P, D, P)$.
- Una racha de patos, contada por $f(P, D, D)$.

Estas clases no se superponen y contienen todas las filas válidas. Por lo tanto, su suma es la respuesta buscada. Si no hay tropas, existe solo la fila vacía y la respuesta es 1.

Superposición de subproblemas

Un estado de la función final queda determinado por:

- La cantidad n_P de patos restantes: $P + 1$ posibilidades.

Superposición de subproblemas

Un estado de la función final queda determinado por:

- La cantidad n_P de patos restantes: $P + 1$ posibilidades.
- La cantidad n_D de dodos restantes: $D + 1$ posibilidades.

Superposición de subproblemas

Un estado de la función final queda determinado por:

- La cantidad n_P de patos restantes: $P + 1$ posibilidades.
- La cantidad n_D de dodos restantes: $D + 1$ posibilidades.
- El tipo de la última racha *ultTropa*: 2 posibilidades.

Por lo tanto, hay a lo sumo

$$2(P + 1)(D + 1)$$

estados, es decir, $O(PD)$ cuando $P, D > 0$.

Superposición de subproblemas

Un estado de la función final queda determinado por:

- La cantidad n_P de patos restantes: $P + 1$ posibilidades.
- La cantidad n_D de dodos restantes: $D + 1$ posibilidades.
- El tipo de la última racha *ultTropa*: 2 posibilidades.

Por lo tanto, hay a lo sumo

$$2(P + 1)(D + 1)$$

estados, es decir, $O(PD)$ cuando $P, D > 0$.

Por ejemplo, con $P = D = 4$, $m_P = 3$ y $m_D = 2$, los prefijos $P^1 D^2 P^3$ y $P^2 D^2 P^2$ llegan ambos al estado $f(0, 2, P)$. Sin memoización, ese mismo subproblema puede calcularse más de una vez.

Error frecuente

Un ejemplo de repetición ilustra la superposición, pero para justificar su impacto hay que comparar la cantidad de llamados con la cantidad de estados.

Cantidad de llamados sin memoización

Cada llamada agrega una tropa, por lo que la profundidad es a lo sumo $P + D$. En cada estado hay como máximo dos opciones: poner un pato o poner un dodo.

Cantidad de llamados sin memoización

Cada llamada agrega una tropa, por lo que la profundidad es a lo sumo $P + D$. En cada estado hay como máximo dos opciones: poner un pato o poner un dodo.

Para ver el crecimiento exponencial, tomemos $P = D = q$ y $m_P, m_D \geq 2$. Agrupamos la fila en q pares; cada par puede ser PD o DP . Todas las elecciones forman filas válidas distintas, así que hay al menos esta cantidad de hojas en el árbol de llamadas:

$$2^q = 2^{(P+D)/2}$$

Cantidad de llamados sin memoización

Cada llamada agrega una tropa, por lo que la profundidad es a lo sumo $P + D$. En cada estado hay como máximo dos opciones: poner un pato o poner un dodo.

Para ver el crecimiento exponencial, tomemos $P = D = q$ y $m_P, m_D \geq 2$. Agrupamos la fila en q pares; cada par puede ser PD o DP . Todas las elecciones forman filas válidas distintas, así que hay al menos esta cantidad de hojas en el árbol de llamadas:

$$2^q = 2^{(P+D)/2}$$

En esta familia, los estados son $O(PD) = O(q^2)$, pero los llamados son exponenciales. Por eso $PD \ll 2^q$ para q suficientemente grande y la memoización evita mucho trabajo repetido.

La cota depende de la instancia

No todas las entradas tienen dos ramas en cada nivel: si se agota un tipo o se alcanza un máximo, una opción desaparece. La familia balanceada muestra el crecimiento exponencial que motiva la memoización.

Algoritmo top-down

La función principal consulta la memoria y suma las dos clases de elección: agregar una racha de patos o una de dodos.

Algoritmo top-down

La función principal consulta la memoria y suma las dos clases de elección: agregar una racha de patos o una de dodos.

```
1 PATO, DODO = 0, 1
2
3 def f(np, nd, ultima):
4     if np == 0 and nd == 0:
5         return 1
6
7     if memoria[np][nd][ultima] != -1:
8         return memoria[np][nd][ultima]
9
10    combinaciones = (
11        ponerPATO(np, nd, ultima)
12        + ponerDODO(np, nd, ultima)
13    )
14
15    memoria[np][nd][ultima] = combinaciones
16    return combinaciones
```

Función auxiliar: colocar patos

Solo podemos colocar una racha de patos si la última racha no fue de patos.

Función auxiliar: colocar patos

Solo podemos colocar una racha de patos si la última racha no fue de patos.

```
1 def ponerPATO(np, nd, ultima):
2     combinaciones = 0
3
4     if ultima != PATO:
5         for i in range(1, min(np, MP) + 1):
6             combinaciones += f(
7                 np - i, nd, PATO
8             )
9
10    return combinaciones
```

Función auxiliar: colocar dodos

Simétricamente, solo podemos colocar una racha de dodos si la última racha no fue de dodos.

Función auxiliar: colocar dodos

Simétricamente, solo podemos colocar una racha de dodos si la última racha no fue de dodos.

```
1 def ponerDODO(np, nd, ultima):
2     combinaciones = 0
3
4     if ultima != DODO:
5         for i in range(1, min(nd, MD) + 1):
6             combinaciones += f(
7                 np, nd - i, DODO
8             )
9
10    return combinaciones
```

Memoria e inicialización

La memoria tiene una posición por cada par (np, nd) y por cada posible tipo de última racha.

Memoria e inicialización

La memoria tiene una posición por cada par (np, nd) y por cada posible tipo de última racha.

```
1 memoria = [  
2     [  
3         [-1 for _ in range(2)]  
4         for _ in range(D + 1)  
5     ]  
6     for _ in range(P + 1)  
7 ]  
8  
9 if P == 0 and D == 0:  
10     respuesta = 1  
11 else:  
12     respuesta = (  
13         f(P, D, PATO)  
14         + f(P, D, DODO)  
15     )
```

Complejidad temporal

La complejidad es la cantidad de estados calculados multiplicada por el costo de resolver cada estado.

Complejidad temporal

La complejidad es la cantidad de estados calculados multiplicada por el costo de resolver cada estado.

Hay a lo sumo

$$2(P + 1)(D + 1)$$

estados. Sea

$$R = \max\{\min(P, MP), \min(D, MD)\}.$$

En cada estado se prueban como máximo R longitudes de racha; incluimos el costo constante incluso si la suma queda vacía. La complejidad temporal es

Complejidad temporal

La complejidad es la cantidad de estados calculados multiplicada por el costo de resolver cada estado.

Hay a lo sumo

$$2(P + 1)(D + 1)$$

estados. Sea

$$R = \max\{\min(P, MP), \min(D, MD)\}.$$

En cada estado se prueban como máximo R longitudes de racha; incluimos el costo constante incluso si la suma queda vacía. La complejidad temporal es

$$O((P + 1)(D + 1)(R + 1)).$$

Complejidad espacial

La tabla de memoización tiene dimensiones

$$(P + 1) \times (D + 1) \times 2,$$

por lo que ocupa $O((P + 1)(D + 1))$ espacio.

Complejidad espacial

La tabla de memoización tiene dimensiones

$$(P + 1) \times (D + 1) \times 2,$$

por lo que ocupa $O((P + 1)(D + 1))$ espacio.

Cada llamada recursiva reduce $np + nd$ al menos en uno. La pila tiene profundidad a lo sumo $P + D + 1$, es decir, $O(P + D)$ espacio adicional.

Tips - I

- Resolver ejemplos pequeños a mano ayuda a identificar qué información debe representar cada estado.
- Expliquen la semántica de la función y el significado de cada parámetro.
- Revisen los casos base, los estados imposibles y las restricciones de cada transición.
- Para analizar la complejidad, cuenten los estados y calculen el costo de resolver cada uno; compárenlo con backtracking.

Tips - II

- En los DAGs, un orden topológico permite calcular los estados en un orden compatible con sus dependencias.
- En problemas de optimización con más de un criterio, como OptiPago, definan explícitamente cómo se comparan las soluciones.
- En problemas de conteo, como las filas de tropas, asegúrense de que las elecciones consideradas sean disjuntas y cubran todos los casos.
- Para justificar superposición de subproblemas, comparen cotas de cantidad de llamadas y de estados; no alcanza con un ejemplo puntual.
- Antes de implementar, prueben la recurrencia con instancias pequeñas y casos límite.

Referencias externas

- El ejército del rey: King Army (Codeforces 72G).
- Las legiones del César: Codeforces 118D.